

Motivation

Filter written in Futhark

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

Example

Example using an is even predicate.

as $:= [0, 1, 2, 3, 4, 5]$

Example

Example using an is even predicate.

```
as := [0, 1, 2, 3, 4, 5]  
flags := [t, f, t, f, t, f]
```

Example

Example using an is even predicate.

```
as := [0, 1, 2, 3, 4, 5]  
flags := [t, f, t, f, t, f]  
offsets := [1, 1, 2, 2, 3, 3]
```

Example

Example using an is even predicate.

```
as := [0, 1, 2, 3, 4, 5]
flags := [t, f, t, f, t, f]
offsets := [1, 1, 2, 2, 3, 3]
is := [0, -1, 1, -1, 2, -1]
```

Example

Example using an is even predicate.

```
as := [0, 1, 2, 3, 4, 5]
flags := [t, f, t, f, t, f]
offsets := [1, 1, 2, 2, 3, 3]
is := [0, -1, 1, -1, 2, -1]
result := [0, 2, 4, 0, 0, 0]
```

Example

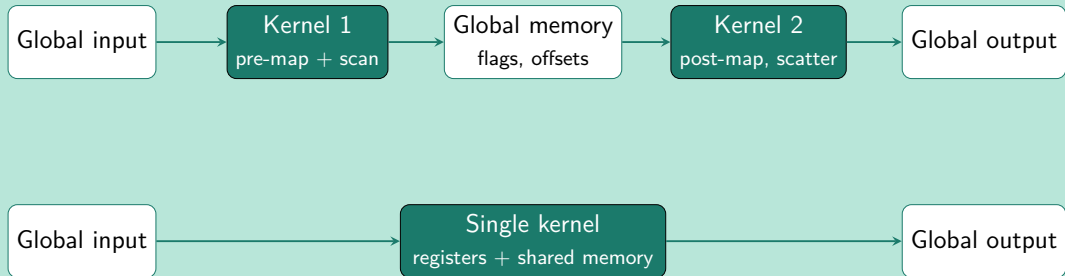
Example using an is even predicate.

```
as := [0, 1, 2, 3, 4, 5]
flags := [t, f, t, f, t, f]
offsets := [1, 1, 2, 2, 3, 3]
is := [0, -1, 1, -1, 2, -1]
result := [0, 2, 4, 0, 0, 0]
take offsets[n - 1] result := [0, 2, 4]
```

Filter written in Futhark

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

Filter written in CUDA



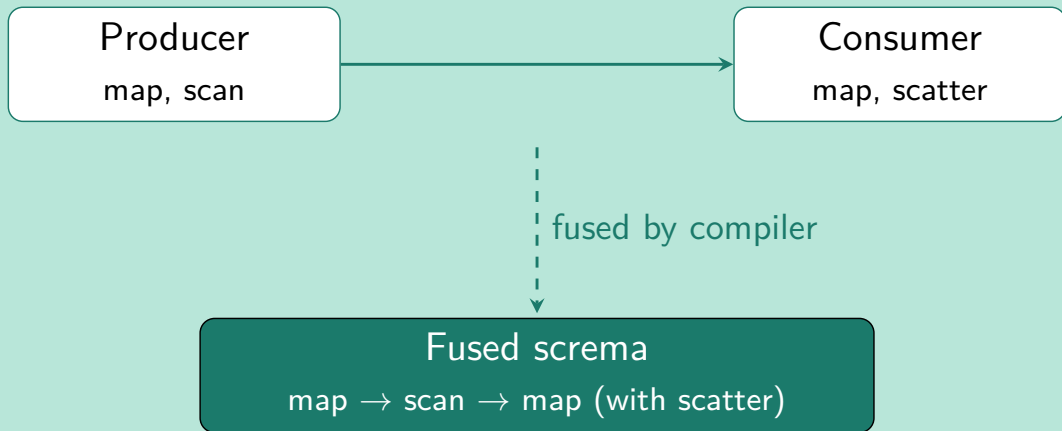
- Streaming Scan.
- Avoid global memory round-trip.
- Not the only optimization.

Levels of Optimization

Method	Full	Dense	Moderate	Sparse	Empty
2K	1	1	1	1	1
1K	3.5	3.7	4	4.4	4.9
B	3.5	3.7	4.3	4.9	5.7
CW	3.4	3.6	3.8	4.2	5.7
FSMW	3.5	3.8	4.2	5	5.9

- Input of 500MiB signed 32-bit integers.
- Close to the theoretical bandwidth limit.

Fusion of IR



No dedicated fused primitive — only `map`, `scan`, and `scatter`, combined by fusion.

```
scanscatter  $f \ (\oplus) \ g$  as  $ds =$   
  let  $(bs, gs) = (\text{unzip} \circ \text{map } f) \text{ as}$   
  let  $bs' = \text{scan } (\oplus) \ bs$   
  let  $(is, vs, ss) = (\text{unzip}_3 \circ \text{map}_2 \ g) \ bs' \ gs$   
  let  $ds' = \text{scatter } ds \ is \ vs$   
  in  $(ds', ss)$ 
```

Accumulators

- Write-only array view; updates accumulate in place.
- Uniqueness types: consumed at most once.
- $[](\text{acc } \tau) \cong \text{acc } \tau$ — so `map` can take and return accumulators.

```
with_acc : *[] $\tau$   $\rightarrow$  (acc  $\tau \rightarrow$  acc  $\tau$ )  $\rightarrow$  [] $\tau$   
update_acc : *acc  $\tau \rightarrow$  int  $\rightarrow$   $\tau \rightarrow$  acc  $\tau$ 
```

Scatter, expressed via accumulators:

```
with_acc ( $\lambda \text{acc} \rightarrow \text{map}_2 (\lambda i \ v \rightarrow \text{update\_acc } \text{acc } i \ v) \text{ is } \text{vs}) \text{ ds}$ 
```

```
maposcanomap  $f$  ( $\oplus$ )  $g$   $as$  =  
  let  $(bs, gs)$  = (unzip  $\circ$  map  $f$ )  $as$   
  let  $bs'$  = scan ( $\oplus$ )  $bs$   
  in (map2  $g$ )  $bs'$   $gs$ 
```

Results: Filter

Method	Full	Dense	Moderate	Sparse	Empty
F2K	1	1	1	1	1
F1K	3.5	3.6	4	4.4	4.5

Method	Full	Dense	Moderate	Sparse	Empty
F1K	1	1	1	1	1
CUB	1	1.1	1.1	1.2	1.2

Results: Partition

Method	Full	Dense	Moderate	Sparse	Empty
F2K	1	1	1	1	1
F1K	2.5	2.6	2.6	2.6	2.3

Method	Full	Dense	Moderate	Sparse	Empty
F1K	1	1	1	1	1
CUB	1.4	1.4	1.4	1.4	1.4

Results: Tokenizer

Method	Dense	Moderate	Sparse
F3K	1	1	1
F2K	1.6	1.8	2

Method	Dense	Moderate	Sparse
F2K	1	1	1
2K	1.8	1.2	1.1